MILESTONE – 4
Deployment of ExoHabitAI System

1. Introduction

Deployment is the process of making a developed application available for users through the internet. After developing and testing the ExoHabitAI system, it is deployed on a cloud platform so that users can access it easily.

In this project, the ExoHabitAI application is deployed using cloud hosting services and web frameworks to provide real-time predictions of planet habitability.

---

2. Objectives of Deployment

The main objectives of deployment are:

- To make the application accessible online
- To ensure continuous availability
- To provide scalability
- To maintain system reliability
- To allow users to access the system from anywhere

Deployment helps in transforming a local project into a real-world usable system.

---

3. Tools and Technologies Used

The following tools and technologies are used for deployment:

3.1 GitHub

GitHub is used for version control and project management. All project files are stored in a remote repository.

3.2 Render Cloud Platform

Render is used as a cloud hosting service to deploy the backend and frontend services.

3.3 Streamlit

Streamlit is used to create a web-based interface for interacting with the prediction model.

3.4 Flask

Flask is used for building REST APIs that connect the machine learning model with the frontend.

3.5 Gunicorn

Gunicorn is used as a WSGI server for running Flask applications in production.

3.6 Python

Python is used as the main programming language for backend and machine learning implementation.

---

4. System Architecture

The deployment architecture consists of the following components:

1. User Interface (Streamlit Web App)
2. Backend Server (Flask API)
3. Machine Learning Model
4. Cloud Hosting Platform
5. GitHub Repository

Working Flow:

1. User enters planet parameters
2. Streamlit sends data to Flask API
3. Flask processes data using ML model
4. Prediction is generated
5. Result is returned to user

This architecture ensures smooth communication between components.

---

5. Deployment Process

Step 1: Project Preparation

- All project files are organized properly
- Required dependencies are added in `requirements.txt`

Step 2: Version Control

- Project is uploaded to GitHub
- Regular commits are maintained

Step 3: Cloud Service Setup

- A web service is created on Render
- GitHub repository is connected

Step 4: Configuration

- Root directory is set

- Build commands are configured
- Start commands are specified

Step 5: Dependency Installation

- Render installs packages using:

```
pip install -r requirements.txt
```

Step 6: Application Execution

- Flask and Streamlit services are started
- Server listens on assigned port

Step 7: Testing

- Deployed application is tested
- Errors are resolved

---

6. Requirements File

The `requirements.txt` file contains all dependencies required to run the application.

Example:

```
flask
gunicorn
pandas
numpy
scikit-learn
streamlit
requests
```

This file helps in automatic installation of packages during deployment.

---

7. Challenges Faced During Deployment

During deployment, the following challenges were faced:

- Missing dependency files
- Incorrect file paths
- Port configuration issues
- Module import errors
- Cloud server limitations

These issues were resolved through debugging and configuration changes.

---

8. Advantages of Cloud Deployment

- High availability
- Easy scalability
- Cost-effective
- Automatic updates
- Remote accessibility

Cloud deployment improves system performance and reliability.

---

9. Security Considerations

To ensure security:

- Environment variables are used

- Sensitive data is protected
- Access control is implemented
- Secure communication is maintained

Security is important to protect system integrity.

---

10. Future Enhancements

In future, the deployment can be improved by:

- Using Docker containers
- Adding load balancers
- Implementing CI/CD pipelines
- Improving monitoring systems
- Enhancing UI design

These enhancements will increase system efficiency.

CODES

a. Install Dependencies

pip install -r requirements.txt

---

b. Requirements File

flask
gunicorn
pandas
numpy
scikit-learn

streamlit
requests

---

c. Flask App Initialization

from flask import Flask

app = Flask(__name__)

---

d. Flask Prediction API

@app.route("/predict", methods=["POST"])
def predict():

---

e. Gunicorn Server Command

gunicorn backend.app:app

---

f. Streamlit Run Command

streamlit run streamlit_app.py --server.port $PORT --server.address 0.0.0.0

---

g. Streamlit Page Setup

st.set_page_config(page_title="ExoHabitAI", layout="centered")

---

h. API Request from Streamlit

res = requests.post("https://your-url/predict", json=data)

---

i. Git Commands

git add .
git commit -m "Deploy project"
git push origin main

---

j. Render Configuration

Root Directory

backend

Build Command

pip install -r requirements.txt

Start Command

streamlit run streamlit_app.py --server.port $PORT --server.address 0.0.0.0

11. Conclusion

Deployment is a crucial phase in software development. The ExoHabitAI system was successfully prepared for cloud deployment using modern tools and frameworks. Proper configuration and testing ensure smooth operation. This deployment enables users to access planet habitability predictions easily and reliably.